

SQL Injection and Security Review

WordPress shortcode review and secure rewrite

A1. Problems, risks and consequences

Summary: The original shortcode exposes a critical SQL injection risk and bypasses WordPress's supported query APIs. It also has output-safety, reliability, and performance weaknesses.

The two biggest issues

1. SQL injection. User-controlled data from `$_GET['cat']` is inserted directly into the SQL statement. An attacker can manipulate the `cat` parameter to change the query, potentially exposing data or causing damaging database behavior.

2. Direct database querying instead of `WP_Query`. The code manually queries WordPress tables even though `WP_Query` already handles taxonomy filtering, table prefixes, query construction, and standard WordPress behavior. Direct SQL here is more fragile and easier to get wrong.

#	Problem	What is wrong	Actual risk / consequence
1	SQL injection	<code>\$_GET['cat']</code> is concatenated into SQL.	Critical: attacker-controlled input can alter the query and potentially disclose or modify database data.
2	No validation of <code>\$_GET['cat']</code>	The input is trusted without checking its type or presence.	Malformed or malicious values reach the database and can cause errors or unexpected behavior.
3	Hard-coded wp_ table prefix	Uses <code>wp_posts</code> and <code>wp_term_relationships</code> directly.	Breaks on WordPress installations configured with a custom table prefix.
4	Bypasses WordPress APIs	Uses a handcrafted SQL query rather than <code>WP_Query</code> .	Misses WordPress query behavior, extensibility, and maintainability; makes defects more likely.
5	<code>SELECT *</code>	Fetches every post column regardless of need.	Unnecessary database and memory work; the query is less explicit.
6	No post-type restriction	Only <code>post_status</code> is limited.	May return published custom post types associated with the term when only standard posts were intended.
7	No limit or pagination	Every matching row is returned.	Large categories can create expensive queries and excessive HTML output, hurting performance.
8	Unescaped title and content	Database values are emitted into HTML without context-appropriate escaping.	Stored XSS or unsafe markup can reach visitors if content is not properly filtered.
9	Raw <code>post_content</code> output	The content is printed without WordPress's normal content pipeline.	Shortcodes, embeds, formatting, and other content filters may not work correctly.
10	No shortcode attributes	The shortcode relies on the URL query string.	It is less reusable and couples page content to a request parameter rather than <code>[featured cat="123"]</code> .

#	Problem	What is wrong	Actual risk / consequence
11	Potential PHP notice	\$_GET['cat'] is accessed without checking that it exists.	Can emit an Undefined array key notice, especially with debugging enabled.
12	Non-numeric input can produce SQL errors	The taxonomy ID is not coerced to an integer.	Malformed input can create invalid SQL or unpredictable query behavior.
13	No overall output-safety strategy	HTML output is assembled ad hoc.	Makes it easy to introduce context-mismatched escaping or unsafe markup in future changes.
14	Generic function name	get_featured_posts() is not namespaced or prefixed.	Can collide with another theme or plugin function and trigger a fatal Cannot redeclare error.
15	Echoes from a shortcode	Output is sent immediately rather than returned.	Content can appear in the wrong place and interfere with WordPress rendering.
16	No empty-result handling	No feedback is shown when no posts match.	Poor user experience and harder troubleshooting.

A2. Proper rewrite

Use the shortcode as: `[featured cat="123"]` or `[featured cat="123" per_page="5"]`

```
<?php

function wiseguy_get_featured_posts( $atts ) {

    $atts = shortcode_atts(
        array(
            'cat'      => 0,
            'per_page' => 10,
        ),
        $atts,
        'featured'
    );

    $cat      = absint( $atts['cat'] );
    $per_page = absint( $atts['per_page'] );

    if ( ! $cat ) {
        return '<p>No category specified.</p>';
    }

    if ( ! $per_page ) {
        $per_page = 10;
    }

    $query = new WP_Query(
        array(
            'post_type'      => 'post',
            'post_status'    => 'publish',
            'posts_per_page' => $per_page,
            'tax_query'      => array(
                array(
                    'taxonomy' => 'category',
                    'field'    => 'term_id',
                    'terms'    => $cat,
                )
            ),
            'no_found_rows' => true,
        )
    );
};
```

```

if ( ! $query->have_posts() ) {
    return '<p>No featured posts found.</p>';
}

ob_start();

while ( $query->have_posts() ) {
    $query->the_post();
    ?>
    <article class="post">
        <h3><?php echo esc_html( get_the_title() ); ?></h3>
        <div class="post-content">
            <?php
                echo wp_kses_post(
                    apply_filters( 'the_content', get_the_content() )
                );
            ?>
        </div>
    </article>
    <?php
}

wp_reset_postdata();

return ob_get_clean();
}

add_shortcode( 'featured', 'wiseguy_get_featured_posts' );

```

Why this version is safer and better

- **absint()** ensures the category ID and page size are positive integers.
- **WP_Query** replaces raw SQL and lets WordPress safely build the taxonomy query.
- **Correct table prefix** is handled by WordPress automatically.
- **post_type => post** prevents unintended custom post types from being returned.
- **posts_per_page** bounds the result set to protect performance.
- **esc_html()** safely outputs the post title for HTML text context.
- **wp_kses_post() + the_content filter** allows legitimate post markup while restoring normal WordPress content processing.
- **Output buffering and return** follow the correct shortcode contract.
- **Prefixed function name** reduces the likelihood of conflicts with other code.
- **Validation and empty-state messages** make the behavior predictable and easier to diagnose.

Important taxonomy note

The original SQL uses `term_taxonomy_id`; this rewrite uses the public category `term_id`. Confirm which identifier your caller has before swapping them.

Attribution: ChatGPT